MealChain Guardian

Laporan sistem, kasus, dan alur proses

Satu laporan rujukan: apa yang dibangun, data apa yang dipakai, bagaimana sebuah keputusan lahir dan dieksekusi, serta kasus-kasus yang dipakai untuk menguji dan mendemokannya.

Ruang lingkup	Backend FastAPI, frontend Next.js, agen Bedrock AgentCore, jembatan Joule (A2A), mock SAP
Sumber fakta	Kode di working tree, skema Postgres, dan pengukuran langsung 20-21 Oktober 2026
Status	Semua angka pada laporan ini diukur, bukan perkiraan; keterbatasan ditulis di bagian 13

> Cara membaca laporan ini: bagian 2-3 menjelaskan mesin dan datanya; bagian 4-10 adalah tujuh alur proses yang bisa diuji satu per satu; bagian 11-12 memetakan kasus demo dan integrasi SAP/AWS/Joule; bagian 13 menuliskan apa yang belum selesai supaya klaim tidak melebihi bukti.

21 Oktober 2026

Isi laporan

Bagian	Judul	Isi
1	Ringkasan: apa yang sebenarnya dibangun	Angka kunci, prinsip, dan isi basis data saat ini
2	Arsitektur per lapisan	Tujuh lapisan, alur satu permintaan, tiga batas kepercayaan
3	Model data dan tiga lapis penandaan	Tabel kunci dan arti origin, demo_session_id, demo_case
4	Alur 1 - siklus data demo	Kosong, aktif, perpanjang, akhiri, kedaluwarsa, tanpa sesi
5	Alur 2 - pipeline keputusan	Tujuh tahap, tujuh agen, matriks approval, tujuh status, lima bukti
6	Alur 3 - diagnosis pasokan dan rekomendasi	Neraca, kesegaran, keamanan, Safe Delivered Cost
7	Alur 4 - surplus, kekurangan, dan limbah	Alokasi greedy di server, aturan lokasi, waste summary
8	Alur 5 - keamanan pangan dan rantai bukti	Karantina, eksklusi pemasok, fusi bukti, dossier, ledger
9	Alur 6 - pengiriman dan peta	Empat status perjalanan, posisi truk, keputusan visual
10	Alur 7 - pemantauan, KPI, dan jejak agen	Tujuh metrik, state data, observability, urutan nav
11	Empat kasus demo dan apa yang dibuktikan	Termasuk kasus kelima: data demo yang tidak pernah kosong
12	Integrasi SAP, AWS, dan Joule	Empat jalur, batasnya, dan sebelas skill Joule
13	Gerbang mutu, bukti, dan keterbatasan	Hasil tes, apa yang belum selesai, peta berkas kunci
14	Catatan 21 Oktober: Jakarta saja	Dua kartu per kasus, tiga kiriman per komoditas, dua perbaikan data

Laporan ini ditulis untuk dua pembaca sekaligus: juri yang ingin tahu apakah sistemnya benar-benar bekerja, dan orang yang akan merawat kode ini setelah kompetisi. Karena itu setiap klaim diikuti angka yang bisa diukur ulang, dan setiap bagian menyebut apa yang *tidak* dikerjakan.

1 Ringkasan: apa yang sebenarnya dibangun

MealChain Guardian adalah lapisan pengambil keputusan untuk rantai pasok pangan institusional (SPPG). Ia menerima data pasokan, permintaan, harga, batch, dan pengiriman; mendiagnosis ketidakseimbangan; menyusun kandidat tindakan; memverifikasinya lewat aturan yang tidak bisa ditawar; lalu menyerahkan keputusan ke manusia untuk disetujui sebelum dieksekusi ke SAP. Yang dijual sistem ini bukan "chatbot yang pintar", melainkan **jejak keputusan yang bisa diaudit**: setiap angka punya asal, setiap penolakan punya alasan, dan setiap eksekusi punya rantai bukti.

Secara teknis ia terdiri dari lima bagian yang bisa dinyalakan terpisah: API FastAPI 65 endpoint pada 29 router; frontend Next.js 14 (App Router) dengan 11 tab peran; PostgreSQL 16 sebagai satu-satunya sumber kebenaran; tujuh agen spesialis (Strands SDK) di Bedrock AgentCore Runtime yang dipanggil lewat Gateway; dan jembatan A2A di SAP BTP Cloud Foundry yang membuat Joule bisa membaca keadaan sistem tanpa menulis apa pun.

Objek	Jumlah terukur	Arti
Endpoint backend	65 pada 29 router	Seluruh akses data, keputusan, dan aksi lewat API ini
Agen spesialis	7	supply, demand, freshness, safety, price, logistics, evidence
Tipe keputusan	6	regional_balance, price_anomaly, safety_disruption, supplier_exclusion, supplier_exclusion_restoration, supplier_procurement
Status keputusan	7	proposed, verifier_flagged, pending_approval, approved, rejected, executed, review_only
Metrik KPI	7	dengan state available / no_data / unavailable, bukan angka palsu
Skill Joule (A2A)	11	supply_situation ... system_status, pemilihan deterministik
Kasus demo	4 jenis	8 kartu hidup di Jakarta (2 per kasus): rejection-loop-kuota, shortage-darurat, supplier-janggal, gangguan-logistik

Tabel 1 - Benda yang dihitung pada laporan ini.

Isi basis data saat laporan ini ditulis (template demo hidup, sesi berjalan): 10 lokasi, 60 pemasok, 120 batch, 40 catatan pasokan, 280 catatan permintaan, 124 sinyal harga, 70 rencana menu, 70 peristiwa pengiriman, 53 geometri rute jalan. Di atasnya berdiri 35 keputusan (8 kartu kasus hidup di Jakarta), 72 blok ledger, 82 trace agent, 16 PO mock, dan 2 penerimaan barang yang menunggu persetujuan.

Prinsip yang mengikat seluruh desain, dan yang paling sering menentukan bentuk kode: **matematika tidak pernah diserahkan ke LLM**. Biaya, skor, ambang kesegaran, status keamanan, jarak, dan alokasi dihitung oleh fungsi Python murni yang punya tes; model bahasa hanya memilih tool mana yang dipanggil, merangkum hasil, dan menjelaskan. Karena itu seluruh keputusan bisa direproduksi tanpa memanggil model.

Konsekuensi kedua: **tidak ada auto-execute**. Aksi yang mengubah data (transfer, karantina, eksklusi pemasok, pembuatan PO) harus lewat verifier, lalu approval sesuai matriks peran, baru boleh dieksekusi. Konsekuensi ketiga: **verifier selalu mendahului rekomendasi** - kartu tidak pernah menampilkan "disarankan" sebelum audit selesai, dan verdict FLAGGED/UNAVAILABLE menaikkan jumlah approval yang dibutuhkan.

---

2 Arsitektur per lapisan

Pembagian lapisan mengikuti satu pertanyaan: *kalau lapisan ini mati, apa yang masih bisa dibuktikan?* Backend dan Postgres adalah bukti; agen dan Joule adalah presentasi dan orkestrasi. Karena itu seluruh logika bernilai uang dan keselamatan berada di backend, dan agen boleh dimatikan tanpa mengubah satu angka pun.

Lapisan	Teknologi	Peran dan alasan
Antarmuka	Next.js 14 App Router, TypeScript, Tailwind, shadcn/ui, Recharts, MapLibre GL + OSM	11 tab peran; peta memakai geometri rute yang sudah di-cache sehingga demo tetap jalan tanpa internet
API	FastAPI (Python 3.12), async SQLAlchemy, Pydantic	65 endpoint; OpenAPI dipakai Gateway untuk mengubah endpoint menjadi tool MCP
Data	PostgreSQL 16, satu instance	Relasional karena pasokan, permintaan, bukti, dan keputusan saling merujuk
Agen	Strands Agents SDK di Amazon Bedrock AgentCore Runtime + Gateway, model Claude via Bedrock	Tujuh spesialis + supervisor/verifier; AgentCore Runtime tidak punya versi self-hosted, jadi bagian ini wajib di AWS asli
Integrasi SAP	Mock SAP (Docker) + dokumen PO PDF	Eksekusi keputusan berhenti di PO mock: cukup untuk membuktikan loop, tanpa menyentuh sistem produksi
Integrasi Joule	A2A server di SAP BTP Cloud Foundry (Python, read-only)	Membuat Joule bisa menjawab pertanyaan pasokan/KPI/antrean; pemilihan skill deterministik, tanpa kredensial di repo
Auth	JWT + bcrypt, 3 peran inti	sppg_head, sppg_nutritionist, bgn_monitor; cakupan lokasi ditegakkan di API, bukan hanya di UI

Tabel 2 - Lapisan, teknologi, dan alasan pemilihannya.

Alur satu permintaan, dari klik sampai baris database

Frontend (klien) -> JWT bearer -> router FastAPI -> dependency cakupan peran -> service deterministik -> SQLAlchemy -> Postgres; hasil kembali sebagai JSON dengan amplop yang menyebut periode, cakupan, waktu hitung, dan sumber data. Untuk aksi berisiko, jalurnya bercabang: router memanggil verifier (aturan + bukti), menyimpan Decision beserta trace, lalu menunggu approval. Agen tidak berada di jalur kritis ini; ia memanggil endpoint yang sama seperti UI, sehingga tidak ada logika kedua yang bisa menyimpang.

Tiga batas kepercayaan yang dijaga kode

- **Cakupan peran.** Peran SPPG hanya menerima lokasinya sendiri; permintaan lintas lokasi ditolak di API. Ini yang memaksa panel alokasi surplus dipindahkan ke server (bagian 7), karena satu lokasi tidak pernah bisa memuat pasangan surplus dan kekurangan sekaligus.
- **Data demo versus data kanonik.** Baris demo ditandai tiga lapis dan dibersihkan hanya lewat aturan yang menyebut tanda itu; data kanonik hasil seed tidak pernah ikut terhapus (bagian 4).
- **Bukti sebelum klaim.** Endpoint keputusan mengembalikan chain, traces, compliance, dan dossier; angka KPI yang tidak punya sumber ditulis *unavailable* dengan alasan, bukan 0.

3 Model data dan tiga lapis penandaan

Skema dibangun mengelilingi satu pertanyaan audit: *kalau seseorang menanyakan angka ini, ke baris mana ia bisa dilacak?* Karena itu setiap keputusan menyimpan tipe, status, pelaku, verdict verifier, catatan, jumlah, biaya, target lokasi, komoditas, serta penanda asal. Tabel-tabel kunci:

Tabel	Isi	Dipakai oleh alur
locations, suppliers, commodities	Referensi: 10 lokasi, 60 pemasok, komoditas wortel/telur/ayam/beras	semua
supply_records, demand_records, menu_plans	Pasokan, permintaan harian, rencana menu (40 / 280 / 70 baris)	diagnosis, kecukupan, forecast
price_signals, road_routes	Sinyal harga (100) dan geometri rute jalan ter-cache (53)	SDC, deteksi anomali, peta

Tabel	Isi	Dipakai oleh alur
batches, delivery_events	Batch dengan tanggal panen, umur simpan, log suhu; peristiwa kirim (120 / 70)	kesegaran, keamanan, peta
decisions, approvals, agent_traces	Keputusan, persetujuan per peran, jejak tiap stage	pipeline keputusan
evidence, dossier, ledger	Bukti terfusi, dossier keputusan, buku besar hash	rantai bukti
demo_sessions	Jendela demo: mulai, kedaluwarsa, status	siklus demo
supplier_exclusions, quarantine, receipts, purchase_orders	Aksi administratif dan hasil eksekusi	keamanan pangan, SAP

Tabel 3 - Tabel kunci dan alur yang memakainya.

Kenapa ada tiga penanda, bukan satu

Satu kolom penanda tidak cukup karena tiga kebutuhan berbeda bertabrakan: menghapus semua data demo, menghapus hanya data milik sesi berjalan, dan mengenali kasus skrip untuk demo. Solusinya tiga lapis yang saling melengkapi:

Penanda	Arti	Siapa yang mengisi
origin = 'demo' | 'seed'	Semua baris demo, termasuk yang lahir di luar sesi	Seeder demo dan jalur /demo/seed, /demo/refresh
demo_session_id	Baris itu milik sesi demo tertentu	Penjaga flush otomatis saat sesi aktif, plus tandai_sesi() untuk bulk insert
demo_case	Baris itu bagian dari skrip kasus (mis. shortage-darurat)	POST /demo/seed

Tabel 4 - Tiga lapis penandaan data demo.

Kombinasi ini yang membuat pembersihan bisa dibicarakan dengan tepat: "akhiri sesi" menghapus baris ber-stempel; "mode sesi menyala tanpa sesi aktif" juga menyapu baris `origin='demo'` yang belum ber-stempel (celah yang terukur 21 Oktober: 25 keputusan demo tanpa stempel hidup selamanya); dan baris `origin='seed'` tidak pernah disentuh sehingga peta, KPI, dan suite backend tetap punya data untuk diuji.

---

4 Alur 1 - siklus data demo

Aturan pemilik produk: tampilan default **kosong**. Data operasional hanya ada selama satu jendela waktu demo, dan jendela itu harus bisa ditutup rapi. Karena itu siklus demo adalah mesin keadaan, bukan sekadar tombol:

> KOSONG --(Siapkan data demo)--> AKTIF --(Akhiri sesi / kedaluwarsa)--> KOSONG
> AKTIF --(Perpanjang)--> AKTIF dengan tenggat baru; AKTIF --(lewat tenggat)--> EXPIRED, lalu dibersihkan pada pembacaan berikutnya

Langkah	Apa yang terjadi di backend	Efek yang terlihat
Mulai	Semua baris operasional sesi lain dikosongkan, data disemai ulang untuk tanggal acuan, lalu baris tanpa stempel di-stempel ke sesi baru	Peta, kartu, dan KPI terisi; hitungan mundur sesi berjalan
Perpanjang	Tenggat digeser, status tetap aktif	Hitungan mundur berubah, data tidak berubah
Akhiri	Baris ber-stempel dihapus, sesi ditandai expired, lalu baris demo tanpa stempel ikut disapu	Kembali ke 0 baris demo; banner "Data demo sedang kosong" muncul
Kedaluwarsa	Sama dengan Akhiri, dipicu pembacaan status berikutnya (malas, bukan cron)	Tidak ada data yatim kalau sesi ditinggal
Tanpa sesi	sapu_tanpa_sesi() menghapus origin='demo' tanpa stempel, dijeda 10 detik antar-penyapuan	Data dari jalur lama (/demo/seed, /demo/refresh) tidak lagi abadi

Tabel 5 - Lima langkah siklus demo dan efeknya.

Bukti terukur (20-21 Oktober 2026)

Sebelum perbaikan: 25 sesi demo semuanya expired, tetapi 25 keputusan `origin='demo'` dengan `demo_session_id NULL` (21 di antaranya punya `demo_case`) tetap ada karena tidak ada sesi yang bisa ditutup. Sesudah perbaikan, diukur langsung di aplikasi:

Titik ukur	Keputusan demo	Baris demo (semua tabel)	Baris kanonik (seed)
Sebelum sesi dimulai	25	12.590	490
Sesi aktif	sesuai kasus	terisi ulang	490
Sesudah Akhiri sesi	0	0	490

Tabel 6 - Hasil uji ujung-ke-ujung siklus demo lewat endpoint yang sama dengan tombol UI.

Perilaku ini dijaga dua tes yang dijalankan di basis data sekali-pakai (supaya tes yang mengosongkan tabel tidak menyentuh data kerja): satu tes membuktikan bahwa tanpa sesi hanya baris demo yang hilang sementara baris kanonik dan tabel referensi utuh; satu tes lagi membuktikan bahwa sesi yang berakhir menyisakan tepat 0 baris demo. Catatan jujur: penjaga flush menstempel *semua* baris tanpa stempel saat sesi mulai, termasuk baris kanonik yang lahir di tengah sesi; itu perilaku lama dan di luar cakupan perbaikan ini.

Konfigurasi yang menentukan perilaku

- DEMO_SESSION_MINUTES = 120 dan DEMO_SESSION_MAX_MINUTES = 240: jendela normal dan batas perpanjangan.
- DEMO_SESSION_ENFORCE = true: mode sesi ditegakkan; pembersihan agresif hanya aktif di mode ini.
- DEMO_MODE = false: kartu Demo Control tetap tampil karena UI menghitungnya dari peran, bukan dari flag ini.

---

5 Alur 2 - pipeline keputusan

Satu keputusan melewati tujuh tahap. Yang penting bukan namanya, tetapi **siapa yang boleh menghentikan** dan **apa yang dicatat** di setiap tahap: tiga agen punya hak memblokir (freshness, safety, evidence), empat lainnya hanya memberi masukan. Setiap tahap menulis AgentTrace sehingga durasi dan kegagalan bisa dibaca ulang.

Tahap	Agen	Blokir	Keluaran
DETECT	supply	tidak	Baris kandidat dari mesin rekomendasi + sisi surplus/kekurangan
VERIFY	freshness, safety	ya	Skor kesegaran, status keamanan, alasan penolakan bila ada
TRACE	evidence	ya	Bukti terfusi, konsistensi antar-sumber, dossier
PREDICT	demand	tidak	Forecast permintaan dari histori nyata (fallback statistik deterministik)
OPTIMIZE	price, logistics	tidak	Safe Delivered Cost, jarak/durasi, kelayakan rute
DECIDE	supervisor	tidak	Decision berstatus proposed + trace lengkap
APPROVE / EXECUTE	manusia + aksi	ya	Approval sesuai matriks, PO mock, ledger, status executed

Tabel 7 - Tujuh tahap pipeline dan haknya.

Matriks approval: verdict menentukan jumlah manusia

Verifier menghasilkan salah satu dari tiga verdict. Konsisten berarti satu approver cukup (kepala SPPG atau ahli gizi). Flagged dan unavailable menuntut dua approver dari dua peran berbeda. Transfer operasional selalu menuntut dua peran apa pun verdict-nya, sedangkan keputusan administratif (eksklusi pemasok) mengikuti verdict. bgn_monitor bersifat baca-saja dan tidak pernah dihitung sebagai approver.

Verdict verifier	Minimal approval	Peran wajib	Kapan muncul
consistent	1	sppg_head ATAU sppg_nutritionist	Bukti konsisten dan semua hard constraint lolos
flagged	2	sppg_head DAN sppg_nutritionist	Ada ketidaksesuaian bukti, mis. harga 40% di bawah referensi
unavailable	2	sppg_head DAN sppg_nutritionist	Bukti tidak lengkap atau tidak bisa diverifikasi

Tabel 8 - Matriks approval (deterministik, diuji terpisah).

Tujuh status, satu arah yang jelas

proposed -> verifier_flagged / pending_approval -> approved / rejected -> executed, dengan review_only untuk keputusan yang sengaja tidak dieksekusi (mis. temuan yang hanya perlu dicatat). Keputusan yang tidak disentuh sampai tenggat menjadi kedaluwarsa dan tidak bisa dieksekusi lagi. Setiap perubahan status menyisakan jejak: siapa, kapan, dan catatan apa.

Bukti yang menyertai satu keputusan

- **chain** - urutan peristiwa dari deteksi sampai eksekusi, termasuk versi keputusan sebelumnya.
- **traces** - satu baris per stage: agen, durasi, status, dan ringkasan keluaran.
- **compliance** - apakah keputusan memenuhi aturan yang berlaku pada saat itu.
- **dossier** - berkas pemeriksaan: sumber angka, bukti, approval, dan hasil eksekusi.
- **ledger** - entri hash yang bisa diverifikasi ulang lewat GET /ledger/verify.

Karena kelima jalur ini ada, pertanyaan "mengapa sistem menyarankan ini?" punya jawaban yang bisa diperiksa tanpa menjalankan model bahasa lagi - cukup membaca baris yang tersimpan.

---

6 Alur 3 - diagnosis pasokan dan rekomendasi

Alur ini menjawab pertanyaan harian kepala SPPG: *komoditas mana yang kurang, siapa yang bisa memasok, dan berapa biaya amannya?* Semuanya dihitung, tidak ditebak. Empat mesin bekerja berurutan.

Mesin	Masukan	Aturan yang mengikat
Neraca	supply_records, demand_records per lokasi+komoditas	Kecukupan = pasokan / permintaan; status seimbang/surplus/kekurangan mengikuti ambang yang sama dengan peta
Kesegaran	Tanggal panen, umur simpan, log suhu	Skor turun linear terhadap umur; penalti suhu berbobot 0,30. PASS >= 0,85; VERIFY 0,60-0,85; di bawah 0,60 masuk zona risiko
Keamanan	Log suhu vs batas aman komoditas	Log tidak lengkap -> needs_verification; di luar batas >= 30 menit -> fail (tanpa jalur override); 0 menit -> pass
Biaya	Harga beli, jarak, tarif angkut, penanganan, kesegaran, status keamanan	Safe Delivered Cost, lihat rumus di bawah

Tabel 9 - Empat mesin diagnosis.

Safe Delivered Cost: rumus yang tidak boleh diperdebatkan model

total = purchase + transport + handling + expected_loss + freshness_risk + safety_risk
expected_loss = purchase x 0,2 x (1 - freshness)
freshness_risk = purchase x 0,50 bila freshness < 0,60, selain itu 0
safety_risk = purchase x 0,10 bila safety needs_verification, selain itu 0
Status fail tidak pernah dihitung: ia melempar ValueError sebelum masuk perankingan.
Pembulatan HALF_UP 2 desimal per komponen, total = jumlah komponen terbulatkan (breakdown selalu menjumlah persis).

Rumus ini sengaja kaku. Kalau biaya aman bisa "disesuaikan" oleh model, seluruh argumen anti-fraud runtuh; karena itu ia fungsi murni dengan tes, dan alasan penolakan ikut tercatat. Tarif angkut yang dipakai demo: Rp300 per km per ton, penanganan tetap Rp150.000, harga acuan wortel Rp24.900 per kg.

Dari kandidat ke kartu rekomendasi

Mesin mengumpulkan kandidat pemasok per komoditas, menghitung SDC, lalu mengurutkan berdasarkan skor yang menggabungkan biaya, kesegaran, keandalan pemasok, dan risiko. Setiap kartu membawa angka pendukung, alasan pemilihan, serta tombol yang mengarah ke alur keputusan. Keterbatasan yang jujur: ambang lot minimum dan filter biaya overhead belum dipasang, sehingga engine masih mengembalikan sekitar 98 item termasuk lot kecil - angka ini perlu disaring sebelum dipakai sebagai daftar kerja harian.

---

7 Alur 4 - surplus, kekurangan, dan limbah

Panel alokasi surplus awalnya dihitung di klien dari peta neraca. Itu mustahil benar: peta dibatasi cakupan peran, dan satu lokasi tidak pernah memuat pasangan surplus dan kekurangan sekaligus. Terukur saat itu: Kepala Bandung melihat "0 lokasi surplus, 1 lokasi kekurangan", sehingga panel tidak pernah terisi untuk siapa pun yang boleh membukanya. Perbaikan: hitungan dipindahkan ke server dengan aturan yang sama, dan endpoint mengirim **pasangan** hasil olahan (dari, ke, kg, jarak), bukan angka mentah per lokasi.

Aturan	Isi
Urutan pengisian	Kebutuhan terbesar diisi surplus terbesar lebih dulu
Kardinalitas	Satu sumber boleh melayani beberapa tujuan; satu tujuan boleh diisi beberapa sumber
Syarat lokasi	Hanya lokasi yang kedua sisinya punya baris yang boleh menjadi sumber atau tujuan - surplus di atas permintaan yang tidak tercatat bukan fakta
Jarak	Haversine dibulatkan, sama dengan versi klien supaya angka tidak berubah hanya karena pindah sisi hitung
Keluaran	GET /surplus/allocations: pasangan (dari, ke, kg, jarak_km), plus ringkasan

Tabel 10 - Aturan alokasi surplus (murni, tanpa DB, diuji terpisah).

Limbah dan nilai yang bisa diselamatkan

GET /waste/summary merangkum peristiwa pembuangan per komoditas dan alasan, sehingga panel Surplus & Limbah bisa memisahkan "hilang karena rusak" dari "hilang karena kelebihan pasok". Keduanya punya tindakan berbeda: yang pertama menunjuk ke keamanan pangan, yang kedua ke redistribusi. Halaman ini hanya dibuka peran SPPG, sesuai cakupan wilayah yang mereka pegang.

---

8 Alur 5 - keamanan pangan dan rantai bukti

Ini alur yang paling ketat karena menyangkut pangan yang akan dimakan anak sekolah. Tidak ada jalur override: batch yang gagal tidak bisa "dibujuk" lolos oleh skor biaya yang lebih baik.

Langkah	Endpoint	Yang dijaga
Evaluasi kesegaran	POST /freshness/evaluate	Skor dari umur dan log suhu; penalti suhu 0,30
Evaluasi keamanan	POST /safety/evaluate	needs_verification bila log tidak lengkap; fail bila excursion >= 30 menit
Karantina batch	POST /batches/{id}/quarantine	Batch keluar dari kandidat, alasan tersimpan
Eksklusi pemasok	POST /supplier-exclusions/proposals	Membuat Decision bertipe supplier_exclusion, memakai verifier dan approval yang sama
Pemulihan pemasok	POST /supplier-exclusions/{id}/restore	Kebalikan eksklusi, tetap butuh bukti dan approval
Fusi bukti	POST /evidence/fuse	Menggabungkan beberapa sumber; dua salinan jenis sama bukan dua sumber
Penerimaan barang	POST /receipts, /receipts/{id}/approve|reject	Barang datang diperiksa terhadap PO dan batch
Dossier & kepatuhan	GET /decisions/{id}/dossier, /compliance	Satu berkas yang bisa dibaca auditor
Verifikasi ledger	GET /ledger/verify	Membuktikan tidak ada entri yang diubah diam-diam

Tabel 11 - Sembilan langkah keamanan pangan dan buktinya.

Ada satu keputusan desain yang layak disebut: eksklusi pemasok diperlakukan sebagai workflow administratif lokal, bukan transfer jumlah atau biaya palsu. Payload kandidat wajib membawa batch dan pemasok, keputusan lama tanpa binding unik ditolak, dan saat eksekusi yang dikunci adalah baris pemasok - bukan baris eksklusi yang mungkin belum ada. Tujuannya satu: tidak ada jalan pintas yang membuat "blokir pemasok" berubah menjadi angka yang menguntungkan siapa pun.

Alur bukti juga menutup celah yang sering luput: penilaian kelengkapan bukti hanya menghitung bukti yang lahir *sebelum* approval selesai. Bukti yang dibuat sesudah keputusan disetujui tidak menaikkan skor, karena kalau boleh, siapa pun bisa melengkapi dokumen belakangan lalu mengklaim keputusannya lengkap.

---

9 Alur 6 - pengiriman dan peta

Peta menjawab satu pertanyaan operasional: *kiriman mana yang benar-benar sedang bergerak, dan di mana posisinya sekarang?* Karena itu hanya kiriman berstatus berjalan yang digambar. Aturan ini ditetapkan pemilik produk secara eksplisit: pemasok yang tidak mengirim tidak diberi garis indikator, supaya peta tidak menjanjikan jalur yang tidak ada.

Status perjalanan	Digambar di peta	Ditawarkan di pemilih kiriman
belum_berangkat	tidak	tidak
di_jalan	ya - garis, glint segmen ditempuh, bangun pengiring	ya
sudah_tiba	tidak	tidak
tidak_diketahui	tidak	tidak

Tabel 12 - Empat status perjalanan dan perilakunya di peta.

Bagaimana posisi truk dihitung

Geometri rute diambil sekali saat seeding dari OSRM dan disimpan di road_routes (53 baris), sehingga peta tidak pernah memanggil OSRM dari peramban. Posisi kendaraan = fraksi panjang jalur yang sudah ditempuh, dengan koreksi cos-lat agar jarak di layar tidak melar; arah (bearing) dipakai memutar puck mobil box dari atas sehingga kendaraan terlihat menghadap arah perjalanan. Animasi 700 ms ease-out, satu permintaan animation-frame per perpindahan.

Keputusan visual yang punya alasan

- Pemasok aktif memakai violet #8B5CF6, bukan hijau atau merah: kedua warna itu sudah bermakna aman dan bahaya, dan penanda "sedang mengirim" tidak boleh menyamar sebagai salah satunya.
- Denyut hanya pada pin, tidak pada garis: satu-satunya gerak yang dibenarkan di peta adalah kendaraan, karena posisinya adalah data - sisanya harus tenang.
- Ikon dipisah per makna: SPPG = pin berisi bangunan, pemasok = lingkaran bergaris berisi kardus, kendaraan = puck mobil box dari atas. Satu metafora kendaraan saja, supaya tidak ada dua ikon yang bersaing.
- Pemilih "Pilih kiriman" bisa dilipat bila lebih dari satu baris, dan hanya menawarkan kiriman berjalan; komoditas tanpa kiriman berjalan menampilkan teks apa adanya, bukan pemilih kosong.
- Tombol "Tampilkan semua" diposisikan eksplisit terhadap grup zoom MapLibre (jarak 9 px, target sentuh 44 px), karena tombol ini bukan anggota grup kontrol bawaan.

Hasil ukur pada data demo: 12 baris rute pemasok, hanya komoditas wortel yang punya kiriman berjalan (2 kiriman, CV Hortikultura Subang Lestari menuju Jakarta, ETA sekitar 23.46), satu pin aktif, satu puck mobil. Telur, ayam, dan beras menampilkan "tidak ada kiriman berjalan" dengan nol kendaraan - bukan bug, melainkan keadaan data yang dilaporkan apa adanya.

---

10 Alur 7 - pemantauan, KPI, dan jejak agen

Pemantauan dipisahkan dari operasi harian. Tujuannya bukan menampilkan angka sebanyak mungkin, melainkan menampilkan angka yang *layak dipercaya*: setiap metrik menyebut periode, cakupan, waktu hitung, dan sumber datanya. Kalau sumbernya tidak ada, metrik menulis tidak tersedia beserta alasannya.

Metrik	Definisi	Perilaku saat data tak ada
Meal Continuity Rate	Butuh fakta menu dan pemenuhannya	unavailable - bukan 0, bukan proxy PO
Avoidable Food Loss	Butuh peristiwa pembuangan beserta alasan	unavailable
Regional Imbalance Resolution Rate	Keputusan DETECT akar yang dieksekusi dalam periode	no_data bila tidak ada kasus
Safe Delivered Cost	Rata-rata SDC keputusan yang PO-nya committed pada periode	no_data
Procurement Price Deviation	Rata-rata magnitudo selisih quote terhadap referensi terakhir	no_data
Decision Time	Rata-rata waktu DETECT pertama sampai ACT terakhir yang committed	no_data + coverage
Evidence Completeness	Persentase keputusan dengan >= 2 jenis bukti konsisten sebelum approval	no_data bila belum ada approval

Tabel 13 - Tujuh metrik KPI dan perilakunya ketika data tidak ada.

Jejak agen

Setiap pemanggilan agen menulis catatan observability: agen mana, endpoint mana, berapa lama, berhasil atau gagal, dan ringkasan keluaran. Tab Agent Log membaca catatan itu, sedangkan tab KPI & Kualitas Agen membandingkan hasil agen dengan aturan deterministik - bukan untuk mempercayai agen, melainkan untuk mendeteksi saat agen mulai menyimpang. Di jalur Joule, jembatan A2A memilih skill secara deterministik (JSON eksplisit, nomor PO dalam teks, lalu kata kunci berprioritas), jadi tidak ada pemilihan tool oleh model di jalur baca.

Urutan navigasi yang mengikuti halaman

Nav "Bagian" di Overview sempat terasa melewatkan satu bagian. Penyebabnya bukan gulir, melainkan urutan: nav menaruh Kiriman sebelum Kecukupan, sedangkan halaman merender Kecukupan lebih dulu. Perbaikannya

menyelaraskan nav ke urutan render (Ringkasan, Peta & antrean, Kecukupan, Kiriman, Telemetri) dan menambahkan tes regresi yang membaca id bagian langsung dari halaman, sehingga nav tidak bisa menyimpang lagi tanpa tes merah.

11 Empat kasus demo dan apa yang dibuktikan

Kasus demo bukan cerita hiasan; masing-masing menguji satu mekanisme yang berbeda, dan semuanya disemai oleh POST /demo/seed dengan status akhir yang dijaga tetap hidup minimal 60 menit supaya tidak mati di tengah presentasi. Angka di dalamnya konsisten dengan seed sintetis (telur, SDC Rp16,2 juta, kesegaran 0,90).

Kasus	Alur yang diuji	Status akhir	Yang dibuktikan
rejection-loop-kuota	v1 ditolak karena kuota, v2 ditolak karena harga, v3 diajukan ulang dan siap disetujui	v3 pending_approval	Sistem tidak menyerah pada penolakan: setiap penolakan meninggalkan alasan, dan pengajuan ulang membawa riwayat
shortage-darurat	Kekurangan Jakarta 780 kg dengan jendela keamanan 30 menit tercatat di trace	pending_approval	Keputusan sensitif waktu tetap menyimpan bukti dan tenggat; TTL kartu diperpanjang agar tidak kedaluwarsa saat demo
supplier-janggal	Harga pemasok 40% di bawah referensi	verifier_flagged	Anti-fraud: harga terlalu murah bukan rezeki, melainkan sinyal; butuh dua approver dari dua peran
gangguan-logistik	v1 ditolak karena rute Bogor banjir, v2 memakai sumber alternatif Cianjur	v2 pending_approval	Rencana bisa gagal dan diganti tanpa kehilangan jejak; rute alternatif ikut dihitung biayanya

Tabel 14 - Empat kasus demo, status akhir, dan mekanisme yang diuji.

Kasus kelima yang tidak dipanggungkan: data demo yang tidak pernah kosong

Pertanyaan pemilik produk - "kenapa data demo tidak kembali ke 0 ketika sesi tidak dijalankan?" - menjadi kasus uji tersendiri, dan jawabannya arsitektural: baris demo yang lahir di luar sesi tidak punya stempel, sehingga tidak ada yang menghapusnya ketika sesi ditutup. Perbaikannya (bagian 4) menambahkan penyapuan baris demo tanpa stempel saat tidak ada sesi aktif, dengan batas aman `origin='demo'`, sehingga 25 keputusan demo yang tadinya abadi kini hilang pada pembacaan berikutnya, tanpa menyentuh 490 baris kanonik.

12 Integrasi SAP, AWS, dan Joule

Integrasi di sini punya aturan yang jelas: agen dan Joule boleh membaca dan menjelaskan, tetapi eksekusi tetap lewat backend. Dengan begitu tidak ada dua jalur yang bisa mengubah data yang sama.

Jalur	Bentuk	Batas yang dijaga
Backend -> SAP	Mock SAP di Docker; PO dibuat lewat aksi yang sudah disetujui; PDF PO dihasilkan dari data PO	Tidak ada panggilan SAP produksi; nomor PO deterministik sehingga bisa diuji
Backend -> AgentCore	Runtime + Gateway; OpenAPI backend diubah menjadi tool MCP	Agen memanggil endpoint yang sama dengan UI; tidak ada logika kedua
Joule -> backend	A2A server read-only di Cloud Foundry, 11 skill	Tanpa kredensial di repo; origin dibatasi; pemilihan skill deterministik
Tunnel	Cloudflare quick tunnel untuk menjangkau backend dari CF/AWS	Bersifat sementara: mati saat mesin tidur, jadi demo juri butuh named tunnel

Tabel 15 - Empat jalur integrasi dan batasnya.

Sebelas skill Joule

supply_situation, kpi_snapshot, approval_queue, purchase_order, supplier_directory, price_intelligence, demand_forecast, food_loss, supplier_risk, decision_evidence, system_status. Semuanya membaca endpoint cepat tanpa rantai model, sehingga jawaban Joule bisa diulang dan diperiksa. Pemilihan skill memakai urutan tetap: kontrak JSON eksplisit lebih dulu, lalu nomor PO di dalam teks, lalu kata kunci berprioritas - kata kunci yang bertumpang-tindih diperiksa lebih dulu supaya tidak pernah kalah diam-diam.

Di sisi AWS, AgentCore Runtime berstatus READY versi 23 dengan variabel lingkungan yang menunjuk ke tunnel aktif. Enam skenario Joule terbaru belum dideploy ke tenant karena proses deploy menuntut login IAS yang harus diketik manusia; itu dicatat sebagai pekerjaan tersisa, bukan diklaim selesai.

13 Gerbang mutu, bukti, dan keterbatasan

Klaim pada laporan ini berdiri di atas tiga gerbang: tes, pemeriksaan tipe, dan pemeriksaan langsung di aplikasi yang berjalan. Angka tes berubah setiap kali pekerjaan ditambahkan; yang ditulis di bawah adalah hasil terakhir yang benar-benar dijalankan, bukan target.

Gerbang	Perintah	Hasil terakhir
Frontend unit/komponen	npx vitest run	1.704 tes lulus pada 156 berkas
Tipe TypeScript	npx tsc --noEmit	Bersih untuk kode aplikasi; dua galat tersisa di tes alur lokasi (status wajib)
Lint	npx next lint	Bersih
Backend	pytest (DEMO_SESSION_ENFORCE=false)	Suite penuh terakhir: 1.226 lulus, 24 gagal, 20 dilewati. Setelah perbaikan siklus sesi, delapan berkas yang sebelumnya merah lulus saat dijalankan sendiri; sisa 7 kegagalan ada di berkas agenda (4) dan timeseries (3), keduanya menuntut data uji tertentu, bukan kode produk
Sesi demo	pytest tests/test_demo_session.py (DB sekali-pakai)	11 lulus, termasuk dua tes baru siklus demo
Aplikasi	pengukuran di browser	Peta, nav, dan banner demo diukur langsung; 0 error konsol

Tabel 16 - Gerbang mutu dan hasil terakhir.

Yang belum selesai, ditulis apa adanya

- Ambang lot minimum dan filter biaya overhead pada engine rekomendasi belum ada; engine masih mengembalikan sekitar 98 item.
- Enam skenario Joule baru belum dideploy ke tenant (butuh login IAS manual).
- Tunnel Cloudflare bersifat sementara; demo juri sebaiknya memakai named tunnel atau mematikan review multi-agen.
- Penjaga flush menstempel baris kanonik yang lahir di tengah sesi, sehingga sesi berakhir dapat menghapus beberapa baris kanonik (terukur: 2 keputusan).
- Dua galat tipe tersisa berada di tes alur lokasi yang sedang dikerjakan alur lain.
- Commit belum dilakukan karena working tree dipakai bersama beberapa alur pekerjaan.

Peta berkas kunci

Berkas	Peran
backend/app/services/demo_session.py	Siklus sesi, penandaan, dan pembersihan data demo
backend/app/api/demo.py	Seeder empat kasus, refresh, status data
backend/app/core/safe_delivered_cost.py	Rumus SDC dan aturan penolakan
backend/app/core/freshness.py, safety.py	Ambang kesegaran dan status keamanan
backend/app/core/approval_matrix.py	Jumlah dan peran approver per verdict
backend/app/core/allocation.py	Alokasi surplus ke kekurangan (server-side)
agent/hierarchy.json, workers/*	Tujuh agen, tahap, dan hak memblokir
sap-joule/a2a_server/server.py	Sebelas skill Joule dan pemilihan deterministik
frontend/components/SupplyMap.tsx, lib/supplier-routes.ts	Filter kiriman berjalan, posisi truk, garis rute
frontend/lib/section-jump.ts	Urutan navigasi Overview yang mengikuti render halaman

> Cara memakai laporan ini saat presentasi: mulai dari prinsip (bagian 1), tunjukkan satu keputusan utuh lewat chain dan dossier (bagian 5 dan 8), lalu tutup dengan kasus yang gagal secara terkendali - supplier-janggal dan gangguan-logistik (bagian 11). Kegagalan yang terdokumentasi lebih meyakinkan daripada klaim bahwa semuanya selalu benar.

---

14 Catatan 21 Oktober: Jakarta saja

Demo dijalankan sebagai SPPG Jakarta. Karena itu penambahan data pada tanggal ini sengaja dibatasi ke Jakarta; Bandung dan Surabaya dibiarkan seperti sebelumnya supaya angka yang sudah dihafal untuk kota lain tidak bergeser.

Hal	Sebelum	Sesudah (terukur)	Alasan
Kartu kasus hidup di Jakarta	4 (satu per jenis)	8 (dua per jenis)	Satu kartu terlihat seperti kebetulan; dua memperlihatkan pola berulang
Pengiriman per komoditas	tidak merata	3 untuk beras, wortel, telur, ayam	Peta Jakarta butuh cerita pasokan di semua komoditas, bukan satu
Keputusan total	31	35	Empat contoh kedua, satu per jenis kasus
Blok ledger / trace	64 / 74	72 / 82	Ikut bertambah bersama keputusan baru

Tabel 17 - Perubahan data demo untuk Jakarta.

Tiga perbaikan data yang ditemukan saat mengerjakan ini

- **Pemasok tidak sejenis komoditas.** Satu batch beras Jakarta terikat ke pemasok telur, sehingga peta menampilkan "CV Barokah Telur Cianjur" mengirim beras. Diikat ulang ke pemasok beras pada hub yang sama; sekarang 0 baris tidak cocok.
- **Barang gagal uji dihitung sedang dikirim.** Batch beras 800 kg berstatus gagal tetap punya masa layak jauh, jadi muncul sebagai kiriman aktif. Masa layaknya dimundurkan (barang gagal berarti dibuang); kartu Keamanan Pangan tetap menampilkannya sebagai temuan, tetapi peta tidak lagi menggambar garis untuk barang itu.
- **Build berulang menumpuk kasus.** Seeder kasus memakai id acak, jadi setiap build menambah satu set lagi dan jumlah keputusan melonjak. Sekarang kasus Jakarta dibangun ulang dengan paksa sebelum digandakan, dan dua build berturut-turut menghasilkan angka yang sama: 120 batch, 35 keputusan, 72 blok ledger, 82 trace.

Batas dijaga tanpa mengubah definisi "sedang mengirim" yang sudah disetujui pemilik: yang dipangkas adalah baris terkecil, bukan definisinya.